

# Casino Palacio Real

## Identificación de riesgo y probabilidad de respuesta para optimizar la asignación de recompensas

### Informe técnico

Proyecto Integrador · Diploma AI Engineer · DMC Institute

Caso de estudio 2026, datos simulados · Octubre de 2026

Repositorio: [github.com/milagrossanchez/proyecto-integrador-ai-engineer](https://github.com/milagrossanchez/proyecto-integrador-ai-engineer)

## 1. Resumen ejecutivo

El proyecto desarrolla un sistema de recomendación aplicado a la gestión de clientes de un casino ficticio, Casino Palacio Real. A partir del historial de sesiones de juego, el sistema estima el nivel de riesgo de cada cliente (relacionado con juego problemático) y la probabilidad de que responda a distintos tipos de recompensa, y utiliza ambas estimaciones para decidir, bajo un presupuesto fijo, a quién ofrecer qué recompensa — maximizando el beneficio incremental de la campaña sin incentivar el juego problemático.

La solución integra dos bloques del diploma: sistemas de recomendación adaptativos (modelos de riesgo y de propensión, optimización de asignación) y agentes conversacionales con IA generativa (explicación automática de cada decisión, un chatbot con recuperación aumentada (RAG) sobre las políticas del negocio, y un canal de Telegram conectado en vivo). La demostración corre íntegramente en un equipo local, sin dependencias de nube obligatorias.

Con los datos simulados y un presupuesto de referencia de S/ 15 000, el optimizador asigna recompensas a 492 de 888 clientes, con un gasto de S/ 5 205 y un beneficio incremental esperado de S/ 13 242,70. El mismo presupuesto, repartido por una regla de segmento simple (el baseline de comparación), produce un valor esperado negativo de S/ 4 919,66: la regla fija gasta sin discriminar por probabilidad real de respuesta, mientras que el optimizador prioriza a los clientes cuyo beneficio marginal es positivo y mayor cuanto más alto, respetando los límites de riesgo.

El documento describe el problema de negocio, los objetivos y métricas de éxito, los datos y su preprocesamiento, la arquitectura de la solución (la que corre hoy y el objetivo de producción), las decisiones técnicas de implementación, los resultados obtenidos con evidencia reproducible, las limitaciones conocidas y el trabajo futuro.

## 2. Contexto y problema de negocio

Casino Palacio Real es un caso de estudio ficticio que simula un negocio de escala media-alta: aproximadamente US\$ 68 millones de ingresos anuales simulados, con miles de sesiones de juego registradas. Como en cualquier operación de este tipo, el área de marketing dispone de un presupuesto limitado para recompensas (puntos, comps, promociones) y necesita decidir, cliente por cliente, si conviene ofrecerle algo, qué tipo de recompensa y con qué probabilidad de que la campaña realmente genere un retorno incremental.

El problema tiene una restricción adicional que no es solo de rentabilidad: la industria del juego exige un criterio de juego responsable. No basta con maximizar el retorno esperado; el sistema debe excluir explícitamente a los clientes cuyo patrón de juego sugiere riesgo de juego problemático, independientemente de cuán rentable pareciera incentivarlos.

Antes de este proyecto, la asignación de recompensas se resolvía (en el caso de estudio) con reglas fijas por segmento de cliente (VIP, Alto, Medio, Estándar), sin estimar probabilidad de respuesta individual ni diferenciar el efecto causal de la campaña. La sección 8 muestra que esa regla, aplicada a los datos del caso, produce valor esperado negativo: gasta presupuesto en clientes que probablemente no iban a responder, o que habrían generado el mismo valor sin recibir nada.

### 2.1 Justificación del uso de inteligencia artificial

- El riesgo y la probabilidad de respuesta no son observables directamente: se infieren de patrones de comportamiento (frecuencia, duración, volatilidad del resultado, tendencia reciente de actividad), un problema de clasificación/regresión que se presta naturalmente a machine learning supervisado.
- La causalidad importa: una recompensa solo aporta valor incremental si cambia la probabilidad de respuesta respecto a no ofrecer nada. Estimar ese efecto requiere comparar un escenario de tratamiento contra un escenario de control, no solo una probabilidad bruta.
- La comunicación con el cliente y el soporte al operador de marketing se benefician de IA generativa: explicar una decisión automatizada en lenguaje natural, redactar una oferta personalizada y responder preguntas sobre políticas son tareas de generación de texto ancladas en datos estructurados y documentos.

de referencia.

### 3. Objetivos y métricas de éxito

#### 3.1 Objetivo general

Construir un sistema que asigne a cada cliente un nivel de riesgo (Bajo / Medio / Alto) y una probabilidad de respuesta por tipo de recompensa, y que use ambas estimaciones para optimizar la asignación de recompensas bajo un presupuesto fijo, maximizando el beneficio incremental de la campaña sin incentivar el juego problemático.

#### 3.2 Objetivos específicos

- Clasificar a cada cliente en un nivel de riesgo a partir de señales de intensidad de juego, con un modelo supervisado evaluado en un conjunto de prueba independiente.
- Estimar la probabilidad de respuesta a cada tipo de recompensa (baja/media/alta), distinguiendo cuánto de esa respuesta es atribuible a la campaña (efecto causal) y no al comportamiento que el cliente habría tenido de todas formas.
- Optimizar la asignación de recompensas bajo presupuesto, con reglas duras de exclusión por riesgo y topes de concentración por tipo de recompensa y por segmento.
- Explicar cada decisión en lenguaje natural y ofrecer un canal conversacional (web y Telegram) para consultas sobre políticas de recompensas, con una fuente verificable y sin inventar respuestas.
- Documentar honestamente qué está implementado, qué es simulación metodológicamente justificada y qué queda pendiente para producción.

#### 3.3 Métricas de éxito

| Dimensión                    | Métrica                                | Resultado obtenido                                 |
|------------------------------|----------------------------------------|----------------------------------------------------|
| Riesgo                       | F1 macro (holdout)                     | 0,805                                              |
| Respuesta (V1)               | ROC-AUC                                | 0,638 (baseline logística: 0,531)                  |
| Respuesta NBO (V2 calibrado) | ROC-AUC / PR-AUC (test)                | 0,561 / 0,200                                      |
| Valor si responde (V2)       | R <sup>2</sup> (test)                  | 0,604                                              |
| Optimizador                  | Valor esperado vs. baseline por reglas | +S/ 13 242,70 vs. -S/ 4 919,66 (mismo presupuesto) |
| Calidad de software          | Pruebas automatizadas                  | 46/46 (pytest)                                     |

Los valores de ROC-AUC/PR-AUC del modelo de respuesta son modestos en términos absolutos: es consistente con un problema con señal débil (prevalencia de respuesta  $\approx$  14-16% en el histórico simulado) y con la naturaleza semi-sintética del histórico de campañas, discutida en la sección 7 y en las limitaciones (sección 9).

## 4. Datos utilizados y preprocesamiento

### 4.1 Fuente

El dataset base es un CSV simulado de 100 000 sesiones de juego (FctPlayerSession), con una fila por sesión: cliente, máquina, sala, fecha/hora, dinero apostado (CoinIn), resultado (Win), apuesta promedio, tiempo jugado, puntos y comps generados. Se carga en SQL Server 2022 (modelo estrella: dimensiones de cliente, máquina y sala) y, si no hay conexión disponible, el pipeline usa el mismo CSV directamente con un cálculo equivalente en pandas — ambos caminos producen la misma tabla analítica.

### 4.2 Construcción de la tabla analítica por cliente

A partir del detalle de sesiones se agregan 888 clientes con variables de frecuencia (número de sesiones, días activos, sesiones en los últimos 7/90 días), intensidad (CoinIn por hora, apuesta máxima y promedio), duración (promedio y máxima por sesión), comportamiento de riesgo (porcentaje de sesiones largas, de madrugada, de "chasing" — aumentar la apuesta tras una pérdida), volatilidad del resultado, tendencia reciente de actividad y segmento por cuartil de gasto histórico (VIP / Alto / Medio / Estándar).

### 4.3 Etiquetas y evidencia externa

- Riesgo (V1, en producción): etiqueta débil por percentiles sobre las mismas señales de comportamiento, calibrada para aproximar una pirámide realista (≈70% Bajo, 22% Medio, 8% Alto).
- Riesgo (V2, investigación): entrenado sobre un dataset externo público de investigación en juego responsable (12 307 filas, 2073 usuarios, prevalencia de evento de riesgo 7,6%), con un contrato temporal explícito (fecha de observación, fecha de corte, horizonte). No está conectado al scoring en vivo: usa otro espacio de identificadores, y esto se documenta explícitamente para no insinuar una correspondencia falsa con los clientes del caso de estudio.
- Respuesta NBO calibrada (V2): el histórico de campañas es semi-sintético pero sus tasas medias de respuesta están ancladas en evidencia real — el experimento controlado de email marketing de Hillstrom (grupo de control vs. dos tratamientos) y el dataset de uplift de Criteo (tratamiento vs. control). La heterogeneidad individual (afinidad de cada cliente a cada tipo de recompensa) se deriva de atributos propios del cliente, nunca de datos posteriores a la campaña.

Ningún dato semi-sintético se presenta como evidencia real: su función es permitir validar el pipeline completo y la lógica de decisión antes de contar con campañas reales del casino, y se documenta explícitamente en el código y en la documentación técnica qué partes deberán recalibrarse cuando existan esos datos.

## 5. Arquitectura de la solución

Existen dos arquitecturas documentadas, deliberadamente distintas: la que corre hoy para la demostración (local, en una sola computadora) y el objetivo de producción (distribuido, pensado para tráfico masivo de clientes reales). La sección 8 explica qué cambia entre una y otra, y por qué el código actual ya soporta esa transición.

### 5.1 Arquitectura local (demo)

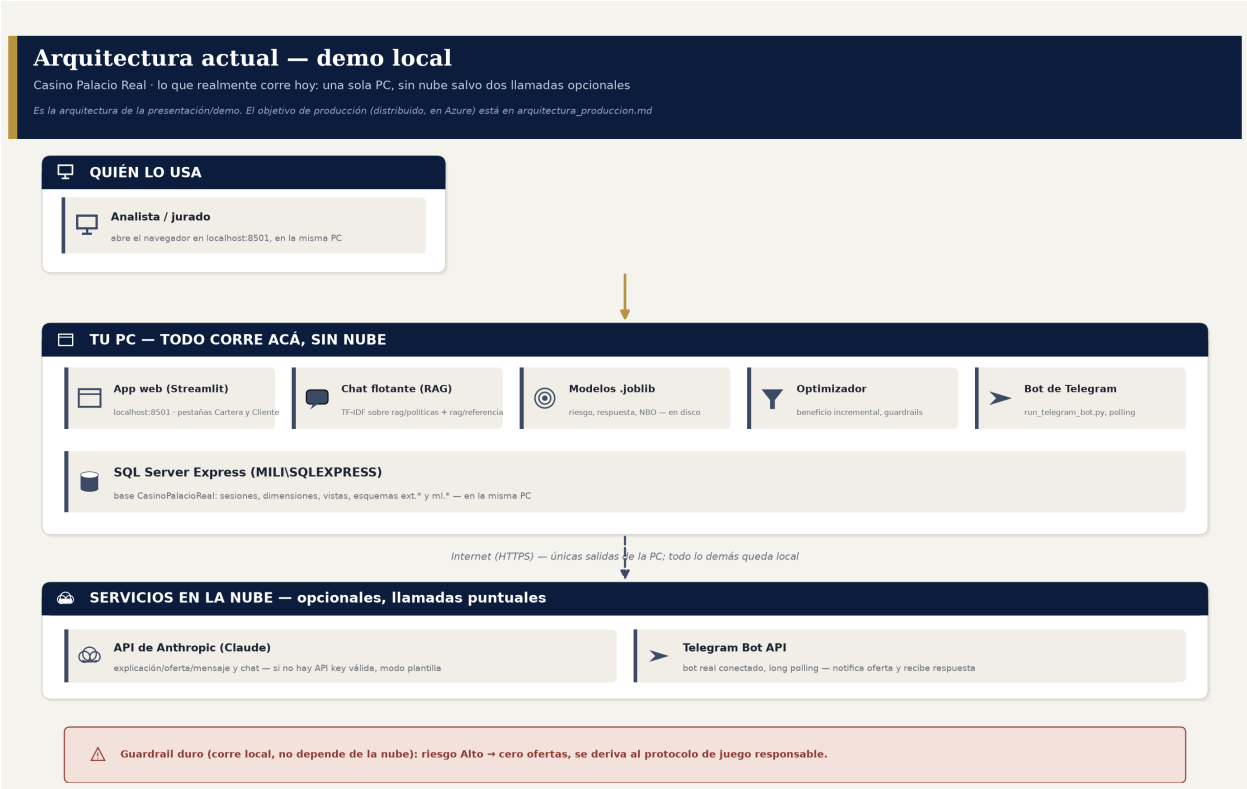


Figura 1. Arquitectura local: todo corre en una sola PC, sin nube salvo dos llamadas opcionales (API de Anthropic y API de Telegram).

La aplicación Streamlit, los modelos (.joblib), el optimizador y SQL Server Express corren en la misma computadora. El chat flotante recupera contexto con TF-IDF sobre documentos de política, sin índice vectorial ni base de datos vectorial. Dos integraciones externas son opcionales: la API de Anthropic (Claude) para explicaciones y chat con lenguaje natural, y la API de Telegram para notificar ofertas y recibir respuestas por ese canal — ambas con una ruta de respaldo (modo plantilla y modo dry-run, respectivamente) que evita que el sistema falle si las credenciales no están disponibles.

### 5.2 Arquitectura objetivo de producción

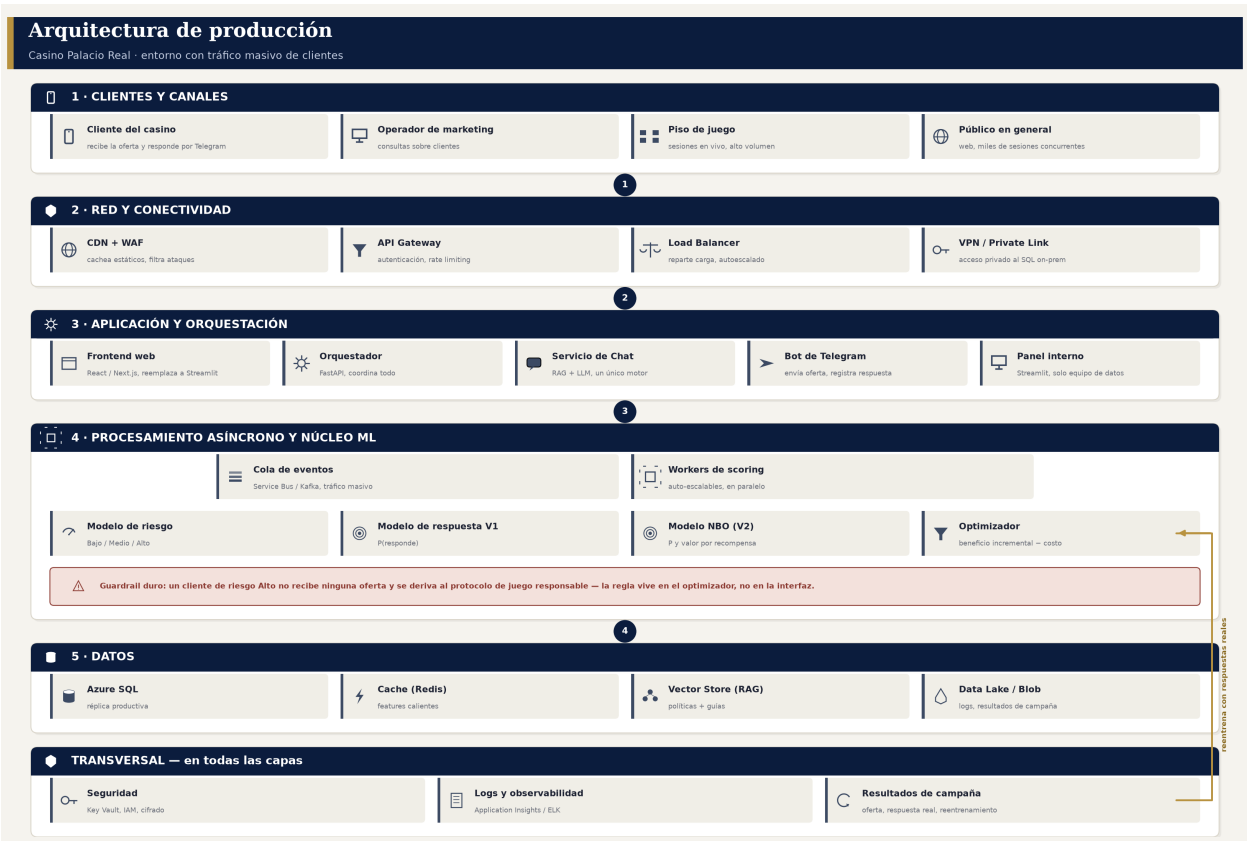


Figura 2. Arquitectura de producción: orquestador, cola de eventos, autoescalado y Telegram por webhook, para tráfico masivo de clientes.

El cambio principal es la introducción de un orquestador (FastAPI o Azure Functions) que coordina modelos, optimizador y notificaciones sin contener lógica de negocio propia; una cola de eventos y workers autoescalables para absorber el tráfico en ráfagas del piso de juego; Azure SQL con caché Redis para features de alta frecuencia; y Telegram migrando de sondeo (long polling, el método usado en la demo) a webhook HTTPS detrás de un API Gateway. Streamlit deja de ser la interfaz pública y pasa a ser el panel interno del equipo de datos.

## 6. Implementación

### 6.1 Bloques del diploma integrados

- Sistemas de recomendación adaptativos: modelos de riesgo y de propensión (respuesta por recompensa), optimización de asignación bajo presupuesto y guardrails.
- Agentes conversacionales con LLM y RAG: chatbot de políticas con recuperación por TF-IDF y generación anclada al contexto recuperado (Claude, vía la API de Anthropic), expuesto en la web y en Telegram.

### 6.2 Tecnologías

| Capa                    | Tecnología                                                                                                          |
|-------------------------|---------------------------------------------------------------------------------------------------------------------|
| Datos                   | SQL Server 2022 (pyodbc / SQLAlchemy); fallback a CSV/pandas                                                        |
| Modelado                | scikit-learn: HistGradientBoostingClassifier/Regressor, CalibratedClassifierCV, IsolationForest, RandomizedSearchCV |
| Optimización            | Python puro (ranking greedy tipo mochila)                                                                           |
| IA generativa           | API Anthropic (claude-sonnet-5); RAG propio (TF-IDF, scikit-learn)                                                  |
| Canal Telegram          | API HTTP de Telegram (long polling en la demo)                                                                      |
| Demo                    | Streamlit                                                                                                           |
| Interfaz administrativa | FastAPI + React / Vite / TypeScript                                                                                 |
| Calidad                 | pytest (46 pruebas), ruff                                                                                           |

### 6.3 Decisiones técnicas clave

| Decisión                                                    | Justificación                                                                                                                                                                                                                                                                                                                                     |
|-------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Gradient boosting en los tres modelos supervisados          | Buen desempeño en datos tabulares; maneja no linealidades e interacciones sin ingeniería manual de features.                                                                                                                                                                                                                                      |
| Calibración de probabilidades (isotónica / sigmoide)        | El optimizador necesita probabilidades realistas, no solo un ranking: decide con valor esperado en soles, no con un score.                                                                                                                                                                                                                        |
| Optimización greedy tipo mochila                            | Solución en milisegundos, fácil de explicar y de auditar (MotivoDecision trazable por cliente), frente a un solver exacto que añadiría complejidad sin cambiar la decisión en la práctica para este tamaño de problema.                                                                                                                           |
| Beneficio incremental (causal) como criterio de ranking     | Evita asignar recompensa a un cliente que probablemente habría respondido igual sin campaña: se prioriza $(P(\text{responde} \text{recompensa}) - P(\text{responde} \text{control})) \times \text{valor} - \text{costo}$ en vez de $P(\text{responde}) \times \text{valor} - \text{costo}$ cuando el modelo con grupo de control está disponible. |
| RAG por TF-IDF en vez de embeddings/vector store            | Suficiente para 6 documentos de política en la demo; documentado explícitamente como la pieza a reemplazar por un índice vectorial en producción.                                                                                                                                                                                                 |
| Streamlit para la demo, FastAPI + React para administración | Streamlit permite construir y demostrar rápido; la API FastAPI prueba, con el mismo código de negocio, que la lógica no está atada a la interfaz — un paso concreto hacia producción, no solo planeado.                                                                                                                                           |

| Decisión                                                     | Justificación                                                                                                                                                                                      |
|--------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Guardrail de riesgo como regla dura en el optimizador</b> | La ética del juego responsable no puede depender de que un LLM "decida bien": la exclusión de clientes de riesgo Alto está en el código de decisión, antes de que cualquier canal pueda notificar. |



## 7. Resultados y evaluación

### 7.1 Modelo de riesgo

El modelo de riesgo en producción (HistGradientBoostingClassifier, con búsqueda de hiperparámetros por validación cruzada) obtiene F1 macro 0,805 en el conjunto de holdout, con recall de 0,93 para la clase mayoritaria (Bajo) y de 0,57 para la clase minoritaria (Alto, 8% de la cartera) — el desbalance de clases se compensa con `class_weight='balanced'`. El modelo de investigación (V2, dataset externo) alcanza ROC-AUC 0,790 y PR-AUC 0,226 en su conjunto de prueba, con un error de calibración (ECE) de 0,012, evaluado sobre splits agrupados por cliente para evitar fuga de información.

### 7.2 Modelo de respuesta

El modelo de respuesta V1 obtiene ROC-AUC 0,638, frente a 0,531 de una regresión logística baseline — una mejora moderada pero consistente. El NBO calibrado (V2), evaluado contra un conjunto de prueba separado por cliente, obtiene ROC-AUC 0,561 y PR-AUC 0,200 para la probabilidad de respuesta, y  $R^2$  0,604 para el valor generado si el cliente responde. Estos valores reflejan la dificultad real de predecir una respuesta binaria con prevalencia baja (14-16%) sobre un histórico simulado; se reportan tal como se midieron, sin ajustar la narrativa a un resultado más favorable.

### 7.3 Optimizador de recompensas

Con los 888 clientes del caso de estudio (621 riesgo Bajo, 196 Medio, 71 Alto excluidos por guardrail) y un presupuesto de referencia de S/ 15 000, el optimizador asigna recompensa a 492 clientes (350 baja, 89 media, 53 alta), con un gasto de S/ 5 205 y un beneficio incremental esperado de S/ 13 242,70. El baseline de comparación — asignar por regla fija de segmento (VIP→alta, Medio→baja, etc.), sin estimar probabilidad de respuesta — resulta en 817 clientes elegibles, un gasto de S/ 7 465 y un valor esperado de -S/ 4 919,66: la regla gasta más y pierde dinero, porque no distingue a quién realmente vale la pena ofrecerle algo.

| Criterio                                   | Clientes | Gasto       | Valor esperado |
|--------------------------------------------|----------|-------------|----------------|
| Optimizador (beneficio incremental causal) | 492      | S/ 5 205,00 | S/ 13 242,70   |
| Baseline por reglas de segmento            | 817      | S/ 7 465,00 | -S/ 4 919,66   |

### 7.4 Calidad de software

El repositorio incluye 46 pruebas automatizadas (pytest) que cubren la construcción de features, los tres modelos, el optimizador (incluyendo el caso específico de que el ranking causal no coincide con el ranking por valor bruto), el contrato HTTP de la API y el canal de Telegram en modo de prueba. Las 46 pruebas pasan en el estado actual del repositorio.

## 8. Limitaciones y trabajo futuro

### 8.1 Limitaciones conocidas

- El histórico de campañas para el modelo de respuesta es semi-sintético: las tasas medias están ancladas en evidencia real (Hillstrom/Criteo), pero la correspondencia entre esos tratamientos y las recompensas baja/media/alta del caso de estudio es una hipótesis de simulación, no un hecho observado en Casino Palacio Real.
- El modelo de riesgo supervisado (V2) no está conectado al scoring en vivo: usa un espacio de identificadores distinto al de los clientes simulados, y conectarlo requeriría un mapeo o un dataset de riesgo propio del casino.
- El RAG del chatbot recupera únicamente documentos Markdown de política (TF-IDF, sin embeddings ni índice vectorial) y no consulta la cartera de clientes ni el historial de ofertas.
- Telegram corre hoy por sondeo (long polling) en vez de webhook, y los eventos de oferta pendiente se guardan en memoria del proceso — se pierden si el proceso se reinicia y no son concurrentes entre réplicas.
- No existe todavía un dataset de resultados reales de campaña con volumen suficiente (se recomienda un mínimo de ~30 respuestas por celda canal × tipo de recompensa) para reemplazar la simulación ni para recalibrar los modelos.
- No hay despliegue en contenedores ni infraestructura como código en el repositorio: la reproducibilidad depende de instalar el entorno local siguiendo el README.

### 8.2 Trabajo futuro

- Migrar Telegram de long polling a webhook HTTPS, con validación de secreto y persistencia de eventos pendientes en base de datos.
- Construir el dataset real de resultados de campaña (sección 7 de docs/arquitectura\_produccion.md) y reemplazar la simulación cuando haya volumen suficiente, recalibrando los modelos de respuesta.
- Incorporar un índice vectorial (embeddings) al RAG y ampliar sus fuentes a la cartera de clientes, con los permisos correspondientes.
- Separar la lógica de negocio en un orquestador independiente de Streamlit (ya probado parcialmente con la API FastAPI) y containerizar el despliegue.
- Evaluar el optimizador con métricas de uplift dedicadas (Qini, AUUC) cuando exista un dataset con grupo de control real, y considerar modelos de uplift específicos (T-learner, X-learner) en vez de la diferencia de dos modelos de probabilidad usada hoy.

## 9. Conclusiones

El proyecto demuestra, con datos simulados pero evidencia reproducible, que un sistema de riesgo + propensión + optimización bajo presupuesto puede superar de forma sustancial a una regla de asignación simple: en el caso evaluado, la diferencia entre asignar por beneficio incremental estimado y asignar por regla de segmento es la diferencia entre un valor esperado positivo (S/ 13 242,70) y uno negativo (–S/ 4 919,66) con un presupuesto comparable.

La incorporación de IA generativa no es un agregado cosmético: el chatbot de políticas y la explicación automática de cada decisión están ancladas en los mismos datos y reglas que usa el optimizador, con una ruta de respaldo explícita (modo plantilla) cuando no hay acceso al modelo de lenguaje, de forma que el sistema nunca inventa una respuesta ni deja de funcionar por falta de credenciales.

El mayor valor del proyecto para un entorno real de casino no está solo en los números de la demostración, sino en la arquitectura: la lógica de negocio (modelos, optimizador, RAG) está separada de la interfaz desde el día uno, y la API FastAPI construida sobre esos mismos módulos demuestra en código, no solo en diagrama, que la transición a un frontend de producción no requiere reescribir el núcleo del sistema.

## 10. Anexos

### 10.1 Enlaces

- Repositorio de código: [github.com/milagrossanchez/proyecto-integrador-ai-engineer](https://github.com/milagrossanchez/proyecto-integrador-ai-engineer)
- Video demostrativo: enlace a agregar por el equipo al publicarlo.

### 10.2 Diccionario de datos (resumen)

| Campo                     | Qué significa                                                                                                      |
|---------------------------|--------------------------------------------------------------------------------------------------------------------|
| NivelRiesgo               | Bajo / Medio / Alto — salida del modelo de riesgo.                                                                 |
| ProbRespuesta             | Probabilidad de que el cliente responda a la recompensa evaluada.                                                  |
| UpliftProbabilidad        | ProbRespuesta con la recompensa menos ProbRespuesta en un escenario de control — la parte atribuible a la campaña. |
| ValorIncremental          | Valor económico esperado si el cliente responde.                                                                   |
| ValorEsperado             | Beneficio incremental: criterio de ranking del optimizador.                                                        |
| Asignada / MotivoDecision | Si la recompensa entró en el presupuesto y los topes, y por qué.                                                   |

Diccionario completo, con cómo se calcula cada campo, disponible en la aplicación (expansor "¿Qué significa cada columna?" en las pestañas Cartera y Cliente) y en entregables/Documentacion\_Tecnica\_Proyecto.docx.

### 10.3 Aviso ético

El sistema no debe incentivar el juego problemático. Los clientes clasificados como de riesgo Alto se excluyen de toda campaña de incentivo y se derivan al protocolo de juego responsable. La capa de IA generativa aplica guardrails que impiden recomendar mayor estímulo de juego a esos clientes, y esa regla se aplica en el optimizador — el núcleo de decisión — no en la interfaz.